MiniRAFT

Distributed Real-Time Drawing Board

Complete Presentation & Viva Study Guide

1. What Are We Building?

This project is a collaborative, real-time drawing board — like a simplified version of Figma or Miro — where multiple users can draw on a shared canvas simultaneously and see each other's strokes instantly.

What makes it interesting is not the drawing itself, but the backend: instead of a single server, the drawing data is managed by a cluster of three replica servers that agree on every stroke through a consensus protocol called RAFT. This means the system can survive server crashes, restarts, and code changes without losing any data or disconnecting any users.

The Core Problem It Solves

In a single-server system, if the server crashes, every user gets disconnected and data can be lost. This project solves that with:

- **Fault tolerance:** any one server can die and the system keeps running
- **Consistency:** every user always sees the same canvas, no matter which server processes their stroke
- **Zero downtime:** you can restart or update any server while users keep drawing

Real-World Equivalent

This is exactly how large-scale systems work in production:

Our Project	Real-World Equivalent
Our Project	Real-World Equivalent
3 Replica Nodes + RAFT	etcd cluster inside Kubernetes
Gateway Service	Load balancer / API gateway
Leader Election	Kubernetes control plane leader
Hot-reload via nodemon	Rolling deployment / blue-green upgrade
Log replication	Database write-ahead log
WebSocket broadcast	Figma / Google Docs real-time sync

2. System Architecture

The system has five Docker containers all running on the same Docker bridge network called raft-net.

The Five Components

A. Three Replica Nodes (replica1, replica2, replica3)

These are the heart of the system. Each replica is a Node.js/Express server running the RAFT consensus protocol. They are identical in code — only their environment variables differ (their ID, port, and who their peers are).

Each replica maintains:

- **An append-only log** — every drawing stroke ever committed, in order
- **A current term number** — used to track which election cycle we are in
- **A state** — either Follower, Candidate, or Leader (explained in Section 3)
- **A commit index** — which log entries have been safely confirmed by the majority

B. Gateway Service

The gateway is the only service that browser clients talk to. It sits between the users and the RAFT cluster and does three things:

1. **Manages WebSocket connections** — any number of browser tabs can connect
2. **Discovers who the current leader is** — polls all replicas every 200ms via GET /status
3. **Forwards strokes and broadcasts commits** — sends strokes to the leader, then broadcasts the committed result to all connected clients

> **Key insight**
> Browser clients never talk directly to replicas. They only talk to the gateway. The gateway handles all the complexity of finding the leader and re-routing when it changes.

C. Frontend

A single HTML file served by nginx. Contains the drawing canvas (mouse and touch support), a real-time cluster status panel showing which replica is leader, its term, log length, and commit index, and an event log showing elections and failovers as they happen.

Data Flow — What happens when you draw a line

1	Your mouse moves on the canvas
2	Frontend sends a stroke message over WebSocket to the gateway
3	Gateway forwards it via HTTP POST /replicate to the current leader replica
4	Leader appends it to its own log, then sends POST /append-entries to both followers

5	Followers append it to their logs and respond with success
6	Leader receives majority acknowledgements (at least 2 of 3 nodes) — entry is committed
7	Leader responds to gateway with { committed: true, entry }
8	Gateway broadcasts the committed stroke to ALL connected WebSocket clients
9	Every browser tab renders the stroke on its canvas

Docker & Networking

All five containers share a single Docker bridge network (raft-net). Replicas talk to each other using container hostnames (replica1, replica2, replica3) on their respective ports. The host machine only exposes:

Port	Service
Port 3000	Frontend (nginx serving index.html)
Port 8080	Gateway (WebSocket + HTTP status)
Port 4001	replica1 (HTTP — for direct debugging)
Port 4002	replica2 (HTTP — for direct debugging)
Port 4003	replica3 (HTTP — for direct debugging)

Bind mounts map each replica's local source folder into its container (e.g. ./replica1 → /app). This means editing a file on disk immediately reflects in the container. nodemon watches for changes and restarts the process automatically — this is how hot-reload works.

3. The Mini-RAFT Protocol

RAFT is a consensus algorithm — a method for making a cluster of servers agree on a sequence of values (in our case, drawing strokes) even when some servers fail. It was designed to be easier to understand than the older Paxos algorithm.

One sentence summary
RAFT works by electing one server as the Leader. The Leader is the only one that can accept new data. It replicates data to Followers, and only considers data committed when a majority confirm they have it.

3.1 The Three States

State	Behaviour
FOLLOWER	The default state. Passively receives heartbeats and log entries from the leader. If it stops hearing from the leader, it becomes a Candidate and starts an election.
CANDIDATE	Actively trying to become leader. Increments the term, votes for itself, asks peers for votes. Becomes Leader if it gets majority. Returns to Follower if it discovers a higher term or loses.
LEADER	The only node that accepts writes. Sends heartbeats every 150ms to suppress elections. Replicates log entries to all Followers. Steps down immediately if it sees a higher term from any peer.

3.2 Leader Election — Step by Step

Every follower has an election timeout set to a random value between 500 and 800 milliseconds. The randomness is critical — it prevents all followers from timing out simultaneously and causing split votes.

4. Follower's election timer fires — it has not heard a heartbeat from the leader
5. Follower transitions to Candidate, increments currentTerm, votes for itself
6. Candidate sends `POST /request-vote` to all other replicas with its current term and log state
7. Each peer grants the vote only if: (a) the candidate's term is at least as high as theirs, (b) they haven't voted for someone else this term, and (c) the candidate's log is at least as up-to-date as theirs
8. If Candidate gets 2 or more votes (majority of 3), it becomes Leader
9. New Leader immediately starts sending heartbeats every 150ms to stop other elections

What is a 'term'?
A term is like a generation counter for leadership. Every time an election starts, the term increments. Terms are used to detect stale messages — if a node receives a message from a lower term, it ignores it. If it receives one from a higher term, it immediately steps down to Follower. This prevents old leaders from causing confusion after they recover from a crash.

3.3 Log Replication — Step by Step

Once a Leader is elected, all writes go through it:

10. Gateway sends stroke data to the leader via `POST /replicate`
11. Leader appends the entry to its own log (with current term number and index)
12. Leader sends `POST /append-entries` to both followers with the entry and a prevLogIndex/prevLogTerm for consistency checking
13. Each Follower verifies the entry is consistent with its log, appends it, and responds success
14. When Leader receives success from at least 2 nodes (itself + 1 follower = majority), the entry is committed
15. Leader responds to gateway with committed: true
16. Gateway broadcasts the stroke to all browser clients

3.4 Safety Rules — What Can Never Happen

- **Committed entries are never overwritten** — once an entry has majority acknowledgement, it stays in the log forever
- **Higher term always wins** — any node that sees a higher term immediately steps down and updates its term
- **Split votes must retry** — if no candidate gets majority, all return to Follower and a new random timeout starts
- **Only one leader per term** — you need majority to win, and each node only votes once per term, so two nodes can't both get majority

3.5 Catch-Up Protocol (Restarted Nodes)

When a replica restarts, it starts fresh with an empty log. Here is exactly how it catches up:

17. Restarted node starts as Follower with empty log
18. Leader sends it a `POST /append-entries` with prevLogIndex pointing to an entry the follower doesn't have
19. Follower's consistency check fails — it responds `{ success: false, logLength: 0 }`
20. Leader sees the logLength in the failure response and knows exactly where the follower's log ends. It calls `POST /sync-log` on the follower, sending all committed entries from that index onward
21. Follower receives all missing entries, appends them, updates its commitIndex
22. Follower is now fully in sync and participates normally from the next round

Important distinction

The /sync-log endpoint is called by the LEADER on a FOLLOWER — not the other way round. The leader pushes missing entries to the lagging follower. The follower just receives and applies them.

4. All API Endpoints

Replica Endpoints (Internal RAFT RPCs)

POST /request-vote

Used during elections. A candidate sends this to each peer asking for their vote.

Sent by	Candidate replica
Received by	All other replicas
Request body	{ term, candidateId, lastLogIndex, lastLogTerm }
Response	{ term, voteGranted: true/false }
Vote granted when	term >= currentTerm AND log is up-to-date AND haven't voted for someone else this term

POST /heartbeat

Sent by the leader every 150ms to assert authority and reset follower election timers.

Sent by	Leader replica
Received by	All follower replicas
Request body	{ term, leaderId }
Response	{ term }
Effect	Resets the follower's election timer. If term > currentTerm, follower updates its term.

POST /append-entries

Used to replicate a single log entry from the leader to followers.

Sent by	Leader replica
Received by	Follower replicas
Request body	{ term, leaderId, entry, prevLogIndex, prevLogTerm }
Success response	{ success: true, term }
Mismatch response	{ success: false, logLength: N, term } — triggers catch-up
prevLogIndex/Term	Consistency check — the entry just before this one must match, otherwise the log has diverged

POST /sync-log

Called by the leader on a lagging follower. Pushes all missing committed entries in one call.

Sent by	Leader replica

Received by	Lagging follower
Request body	{ entries: [...], leaderCommitIndex }
Response	{ ok: true }
Triggered by	Leader detecting logLength mismatch in /append-entries response

POST /replicate

Called by the gateway to submit a new stroke. Only the leader handles this — non-leaders return 403.

Sent by	Gateway
Received by	Leader replica only
Request body	{ data: { x1, y1, x2, y2, color, size } }
Success response (200)	{ committed: true, entry }
Not leader response (403)	{ error: 'not leader', leaderId } — gateway uses leaderId hint to re-route
No quorum response (503)	{ error: 'no quorum' } — less than 2 nodes responded

GET /status

Polled by the gateway every 200ms for leader discovery. Also useful for manual debugging.

Response	{ id, state, term, leaderId, logLength, commitIndex }
Used by	Gateway (every 200ms) and the frontend cluster status panel

Gateway Endpoints

WebSocket ws://localhost:8080

Direction	Message Format
Client → Gateway (draw)	{ type: 'stroke', data: { x1, y1, x2, y2, color, size } }
Gateway → All Clients (committed)	{ type: 'stroke', data: { ... } }
Gateway → All Clients (every 200ms)	{ type: 'cluster_state', statuses: [...] }
Gateway → Client (on error)	{ type: 'error', message: '...' }

GET /status (Gateway)

```
{ leader: 'http://replica1:4001', leaderInfo: {...}, clients: 3 }
```

Stroke Coordinate Encoding

All strokes are stored as normalised coordinates in the range [0, 1] relative to canvas dimensions. When rendering, each client scales back to its own canvas size. This guarantees identical visual output across all clients regardless of window size or screen resolution.

```
// Sending (normalise to 0-1):
x1: mouseX / canvas.width,   y1: mouseY / canvas.height

// Receiving (scale to local canvas):
renderX = data.x1 * canvas.width
```

5. Failure Scenarios

The document lists this as a Week 1 deliverable and a key viva topic. Know all of these cold.

Scenario 1: Leader Crashes

What happens	The leader container is stopped (docker stop replica1)
How detected	Followers stop receiving heartbeats. After 500-800ms their election timers fire.
Resolution	One follower becomes Candidate, starts election, wins majority (both remaining nodes = 2 = majority), becomes new Leader
Client impact	Drawing pauses for ~800ms max during election. WebSocket connections to gateway are NOT dropped. Drawing resumes automatically.
Key guarantee	No committed entries are lost — followers already have them all

Scenario 2: Follower Crashes

What happens	A non-leader replica is stopped
System behaviour	Leader + 1 remaining follower = 2 nodes = still majority. System continues normally.
Client impact	None — users notice nothing
Key point	Commits still require majority (2 of 3). With one follower down, leader + surviving follower = 2. Still enough.

Scenario 3: Crashed Node Restarts

What happens	docker start replica1 (or nodemon auto-restart after file edit)
Node starts as	Follower, empty log, term=0
Catch-up trigger	On first AppendEntries, prevLogIndex check fails. Node reports logLength: 0.
Resolution	Leader calls /sync-log on the restarted node, pushing all missing entries. Node syncs fully.
Result	Node is fully in sync, participates normally in next round

Scenario 4: Hot-Reload (File Edit)

What triggers it	Editing any file in replica1/, replica2/, or replica3/ on the host machine

How it works	nodemon detects the file change and sends SIGTERM. Process shuts down cleanly. Container restarts with new code. Node rejoins as Follower and syncs via /sync-log.
Client impact	Zero. The gateway keeps serving other clients. WebSocket connections stay open. Only a brief pause in commits if the reloaded node was the leader (triggers election).
What this simulates	A blue-green rolling deployment — updating one server at a time with zero downtime

Scenario 5: Two Replicas Down Simultaneously

What happens	2 of 3 replicas are stopped
System behaviour	The surviving node cannot form a majority alone. It cannot commit new entries. System becomes unavailable for writes.
Why this is correct	This is the right behaviour. It is safer to refuse writes than to commit without majority and risk data loss or split-brain.
Recovery	When one crashed node restarts, majority (2 of 3) is restored and the system resumes

Scenario 6: Stale Leader Returns

What happens	An old leader (say replica1, term 3) goes down, a new leader is elected (replica2, term 4). Then replica1 comes back online.
How detected	replica1 sends heartbeats with term=3. Every node it contacts has term=4 and rejects it, responding with term=4.
Resolution	replica1 receives term=4 in the response, immediately steps down to Follower, updates its term to 4. It then catches up via /sync-log.
Key guarantee	Two leaders cannot coexist in the same term. Term numbers make it impossible.

Scenario 7: Split Vote

What happens	Two followers time out at almost the same time and both start elections with the same term
Result	Each votes for itself. Each needs 2 votes. Neither gets the other's vote if each receives the request after already voting.
Resolution	Both return to Follower. New random timeouts start. Because timeouts are random (500-800ms), it is very unlikely they tie again. One starts first and wins.
Design choice	The randomised timeout range (300ms spread) makes this extremely rare in practice

6. Cloud Computing Concepts (Viva Topics)

The assignment explicitly lists these as viva topics. Know them well.

6.1 Consensus & Fault Tolerance

Consensus means getting multiple servers to agree on a value, even when some of them fail. RAFT achieves this through:

- **Quorum (majority):** any operation requires agreement from more than half the nodes. In a 3-node cluster, that is 2. This means the system can tolerate 1 failure.
- **Leader-based writes:** routing all writes through one node avoids conflicts and makes ordering easy.
- **Terms:** monotonically increasing term numbers act as a logical clock, letting nodes detect stale messages and old leaders.

Used in production: etcd (Kubernetes), Consul (HashiCorp), CockroachDB, TiKV (TiDB).

6.2 Zero-Downtime Deployments

Our system demonstrates two patterns used in real production deployments:

- **Rolling update:** restart one replica at a time. While one is reloading, the other two maintain quorum and the system stays available.
- **Blue-green:** the new process starts, joins the cluster as a follower, and catches up before the old process exits. There is always at least one healthy node.

nodemon + bind mounts simulates this. In production, Kubernetes rolling deployments do the same thing — spin up the new pod, wait for it to be healthy, then terminate the old one.

6.3 State Replication & Event Ordering

Every stroke is an event appended to a log. Key properties:

- **Append-only:** entries are never modified or deleted, only appended. This prevents conflicts.
- **Ordered:** every entry has a monotonically increasing index. All nodes agree on the order.
- **Replicated:** every committed entry exists on at least 2 of 3 nodes before the client is told it was accepted.

This is identical to how Kafka topics, database write-ahead logs, and distributed streaming systems work.

6.4 Containerisation & Service Isolation

Each service runs in its own Docker container with:

- **Isolation:** containers cannot interfere with each other's processes or memory

- **Reproducibility:** the Dockerfile defines the exact environment, so it runs the same everywhere
- **Service discovery:** containers find each other by hostname (replica1, replica2) on the shared Docker network, not by IP address
- **Bind mounts:** source code is mounted from the host into the container, enabling live editing without rebuilding images

6.5 Real-Time Systems

WebSockets provide a persistent, bidirectional connection between the browser and gateway. Unlike HTTP polling (where the client asks every N seconds), WebSockets allow the server to push data instantly when a stroke is committed.

This is how Figma, Miro, and Google Docs achieve real-time collaboration. The key design is: strokes are only broadcast after they are committed by the RAFT cluster. This guarantees that all clients receive exactly the same strokes in exactly the same order.

6.6 CAP Theorem (Bonus Viva Knowledge)

The CAP theorem states that a distributed system can only guarantee two of: Consistency, Availability, and Partition Tolerance.

- **Our system chooses CP** — Consistency + Partition Tolerance over Availability. When quorum is lost (2 nodes down), we stop accepting writes rather than risk inconsistency.

This is the same choice made by etcd, ZooKeeper, and CockroachDB. Banking systems make this choice because inconsistency is worse than unavailability.

7. Likely Viva Questions & Answers

Q: Why do you need an odd number of replicas?

Because majority requires more than half. With 3 nodes, majority is 2. With 2 nodes, majority is 2 — but if one fails you have no majority and the system stops. With 3 nodes, one can fail and you still have 2 = majority. If you had 4 nodes, you would need 3 for majority — the same fault tolerance as 3 nodes but with more complexity and cost. 3 and 5 are the common choices in production.

Q: What happens if the network is slow and heartbeats are delayed?

A follower might not receive heartbeats in time and start an unnecessary election. This is a false positive. The current leader would eventually receive a request-vote message with a higher term, step down, and a new election would resolve it correctly. This is called a spurious election. RAFT handles it correctly — the new leader will simply have a higher term.

Q: Why does the election timeout need to be random?

If all followers had the same timeout, they would all become candidates at the same time, all vote for themselves, and no one would get a majority (split vote). By making timeouts random across a range (500-800ms), the first node to time out is very likely to win before the others even start their elections.

Q: Can you have two leaders at the same time?

No. This is guaranteed by the combination of: (1) each node votes for at most one candidate per term, and (2) winning requires a majority. Since there is only one majority possible, only one leader can win per term. If an old leader comes back from a partition, it sees responses with a higher term and immediately steps down.

Q: What is the difference between your /heartbeat and /append-entries?

Heartbeats are lightweight pings sent every 150ms to say 'I am still the leader, reset your election timer.' They carry no log data. Append-entries carries an actual log entry and performs a consistency check. In standard RAFT, heartbeats are actually just empty AppendEntries calls — we separated them for clarity and efficiency.

Q: What if a stroke is drawn while an election is happening?

The gateway cannot find a leader (all replicas report follower or candidate state). The gateway sends the client an error message: 'no leader — election in progress.' The stroke is dropped. In a production system you would queue it and retry. Elections last at most 800ms, so the pause is brief.

Q: How does the system guarantee all clients see the same canvas?

Only committed strokes are broadcast to clients, and an entry is only committed when a majority of replicas have it in their log. Since committed entries are never overwritten, and all clients receive broadcasts in the same order from the single gateway, every client's canvas is always a consistent prefix of the committed log.

Q: Why use normalised coordinates for strokes?

A stroke drawn on a 1920x1080 monitor would render in the wrong position on a 1280x800 laptop if we stored pixel coordinates. Normalising to [0,1] range and scaling on receipt means everyone sees the stroke in the same relative position regardless of their screen size.

8. Demo Guide

Assignment requirement

The demo video must be 8-10 minutes and show: drawing from multiple tabs, killing the leader, automatic failover, hot-reloading a replica, consistent canvas after restarts, and system behaviour under chaotic conditions.

Setup — Do This Before You Start Recording

23. Extract the project: tar -xzf miniraft.tar.gz && cd miniraft
24. Build and start everything: docker-compose up --build
25. Wait about 15 seconds for all containers to start and elect a leader
26. Open http://localhost:3000 in four browser tabs — arrange them so all are visible simultaneously
27. Open two terminal windows: one for commands, one for logs: `docker logs -f replica1 & docker logs -f replica2 & docker logs -f replica3`
28. Verify the sidebar shows one node as LEADER and two as FOLLOWER before you start

Demo 1: Multi-User Real-Time Drawing (required)

What to show: All tabs see identical drawing in real time.

29. Draw a line in Tab 1 — show it appears instantly in Tabs 2, 3, and 4
30. Draw in Tab 2 — show it appears in all other tabs
31. Draw simultaneously in multiple tabs
32. **Point to say:** 'Every stroke goes through RAFT consensus — the leader replicates it to followers, gets majority acknowledgement, then the gateway broadcasts it to all clients. All clients see exactly the same committed strokes in the same order.'

Demo 2: Leader Failover (required)

What to show: Killing the leader causes an automatic election and drawing resumes.

33. Point to the sidebar and name the current leader (e.g. 'replica1 is the leader, term 3')
34. Kill it: `docker stop replica1`
35. Watch the sidebar in real time — within 800ms it shows 'electing...' then a new leader
36. Point to the log terminal — show ELECTION_START, BECAME_LEADER events
37. Draw in the canvas — it works again. Users never disconnected.
38. **Point to say:** 'The followers detected a missed heartbeat after 500-800ms. One became a candidate, got both votes from itself and the other follower, and became the new leader. The gateway discovered the new leader within 200ms.'

Demo 3: Node Restart & Catch-Up (required)

What to show: Stopped node restarts, catches up on missed strokes, participates normally.

39. With replica1 still stopped (from Demo 2), draw several strokes
40. Restart it: `docker start replica1`
41. Watch the logs for replica1: `docker logs -f replica1`
42. Show SYNC_LOG_RECV and SYNC_LOG_DONE events — it is receiving missed entries from the leader
43. Show replica1 in the sidebar — it goes from 'unreachable' to 'follower' with the same log length as the others
44. **Point to say:** 'The restarted node had an empty log. Its first AppendEntries from the leader failed the consistency check. The leader detected the gap and pushed all missing entries via /sync-log. The node is now fully in sync.'

Demo 4: Hot-Reload / Zero-Downtime Update (required)

What to show: Editing a source file triggers a container restart with no client disconnects.

45. Make sure all tabs have drawing on the canvas
46. In a terminal, add a comment to a replica file: `echo "// hot-reload test" >> replica2/server.js`
47. Watch: nodemon detects the change, container restarts, node rejoins as follower
48. Clients in all tabs remain connected — the WebSocket to the gateway never dropped
49. Keep drawing — works immediately
50. **Point to say:** 'This simulates a rolling deployment. We updated the code of one server while the cluster kept serving users. nodemon detected the file change, shut down the old process cleanly, and the new process rejoined the cluster. This is exactly how Kubernetes rolling updates work.'

Demo 5: Consistent State After Multiple Failures (required)

What to show: Canvas is identical across all tabs even after chaotic restarts.

51. Draw a distinctive pattern in all tabs — something recognisable like a large X
52. Kill and restart replicas rapidly while continuing to draw:
    - `docker stop replica1 && sleep 2 && docker start replica1 &`
    - `docker stop replica2 && sleep 1 && docker start replica2 &`
53. Keep drawing in all tabs throughout
54. Once all replicas are back, show that all four tabs have identical canvases
55. Check the log lengths in the sidebar — all three replicas show the same logLength and commitIndex
56. **Point to say:** 'Every stroke on screen was committed by a RAFT majority before being broadcast. Committed entries are never lost and never overwritten. The canvas you see is provably consistent across all nodes and all clients.'

Demo 6: Chaotic Conditions / Stress Test (required)

What to show: System self-heals under rapid successive failures.

57. Run this in a terminal while all tabs are open and multiple users are drawing:

```
# Terminal — run all at once
docker stop replica1 && sleep 1 && docker start replica1 &
docker stop replica2 && sleep 2 && docker start replica2 &
# Watch elections fire rapidly in the logs
```

58. Show the log terminal — rapid ELECTION_START, STEP_DOWN, BECAME_LEADER, CATCHUP events
59. Show that the sidebar updates live, showing state changes
60. Drawing may pause briefly during elections but always resumes
61. **Point to say:** 'Higher term always wins. Even under rapid churn, only one node can be leader at any moment. Split-brain is impossible because you need a strict majority to commit anything.'

What to Have Open During the Demo

What	Why
4 browser tabs at localhost:3000	All visible simultaneously — shows multi-user sync
Terminal 1 (commands)	For running docker stop/start commands
Terminal 2 (logs)	docker logs -f replica1 & docker logs -f replica2 & docker logs -f replica3 — shows RAFT events live
Browser sidebar	Shows live cluster state: leader, term, log length per node
localhost:8080/status	Shows gateway view — which replica it believes is leader

Key Things to Say During Demo

- Point to the term number changing after each election
- Point to log lengths syncing after a restart (all three nodes should show the same number)
- Mention that WebSocket clients never disconnected — gateway provides the stable connection point
- Mention that you are only broadcasting after majority acknowledgement — this is what makes the canvas consistent
- When showing hot-reload, say: 'This replica is now running new code. It rejoined the cluster, caught up on missed entries, and is participating in consensus — all while users kept drawing.'

Debugging Commands (Useful During Demo)

```
# Check which node is leader right now
curl http://localhost:4001/status
curl http://localhost:4002/status
curl http://localhost:4003/status

# Check gateway's view
curl http://localhost:8080/status
```

```
# Watch structured logs from all replicas
docker logs -f replica1 & docker logs -f replica2 & docker logs -f replica3

# Trigger hot-reload
echo "// update" >> replica2/server.js

# Kill and restart a specific replica
docker stop replica1 && docker start replica1
```

Final tip

Before recording the demo video, do a full dry run from scratch (docker-compose down -v && docker-compose up --build) to make sure everything starts clean. Know in advance which replica gets elected leader first — it is usually random but you can check with curl.